

2/10/25

TDD - Test Driven Development.

Teoría - Práctico.

- Implementación de una user en el práctico.
- NO es una metodología de prueba
- Es un método para desarrollar software → forma de construir código guiado por pruebas.

NOTA

Requerimiento
↓
Análisis
↓
Diseño
↓
(arquitectura + imp.) Implementación → 33/35%
Caso de PRUEBA ← Prueba/testing → 30/50% del costo total del proyecto { depende del tipo de software que sea.
Despliegue (tiene impacto en la vida de las personas.)

• Caso de Prueba:

Identificación de situaciones concretas + config. de datos de inicio necesarios para identificar si se cumplen o no criterios especificados.

↳ Act. programada cronológicamente tarde.

(No es necesario tener todo el código programado para empezar con las actividades de prueba.)

Diseñar → Probar → Desarrollar.

- se diseñan todo lo que necesito para probar basandome en los requerimientos y diseño los casos de prueba.

- No se estiman los tiempos de testing ni del rediseño (corrección).

Adelantar el diseño de los casos de prueba.

↳ pensar el testing desde el principio

El error humano y ponerle la culpa a los demás es
⊕ humano todavía.

- la calidad del producto es responsabilidad de todos. No de los de testing.

Despliegue → producción, se comienza a usar.

↳ la acción para modificar el software.

Rol → dev ops → el código se construye vos te encargas de su funcionamiento en producción y del mantenimiento.
development operación.

4 niveles de prueba

↳ prueba unitaria (propio desarrollador)
↳ validar y verificar.

1 de las 13 prácticas de extreme programming → 50%

Integración → genera un build.

sistema / versión

UAT → prueba de aceptación de usuario.

↳ user acceptance test.

implementar.

requerimientos
funcionales +
no funcionales

tenidos
en cuenta.

testing
equipo

P.O (usuario)

NOTA

El análisis esta muy apegado al dueño, se forman como una sola cosa.

BDD → desarrollo conducido x comportamiento.

→ Adelantar el trabajo de testing b(+) si se puede.

3 levels

- ↳ No existir código hasta haber escrito un caso q falle
- ↳ Con solo un test q falle es suficiente.
- ↳ No escribir más código del necesario para q el test pase.

test → Cumpla → Falle → Cambio → Fija → mejoras.

Re-factoring

- introducir cambios reduciendo la posibilidad de introducir defectos.
- mejorar la calidad (q se haga mejor).

↳ la suma TPD ↑ mejor la calidad interna del componente.

TPD para → encontrar un código que "huela mal"

cada línea de código q se hace introduce 3 errores

↳ cada vez q se mete mano a un código q volver a probar.

Práctw:

Taxi noble

escribir q quiero probar.

- Precondición (requisito).

def test_nombre -> camino -> resultado (success/fail) {}:

// Precondiciones. → Todo lo que necesito para poder ejecutar la prueba.

// Datos del caso de prueba.

// Resultados.

→ assert: palabra clave q se usa para verificar condiciones en la prueba. Si la condición assert es falsa, la prueba falla.

→ assert all: se tienen q cumplir todas las condiciones para que se cumpla la prueba.

// mensaje final

↳ si paso o no el test.

↳ si todos los assert pasaron.

NOTA

ICS

HOJA Nº 14

FECHA 2/10

pseudocódigo de la US

(para poder seguirlo)

→ tmb tengo q tener las funciones q voy a probar.

Como minimo deben cubiertas las de la Uter.